

Atividade - Introdução ao Spring Security

Neste arquivo você encontra conceitos básicos de configuração do Spring Security em APIs REST utilizando Spring Boot.

1. Explique com suas palavras o que é Spring Security.

R: O Spring Security é um framework utilizado para adicionar segurança em sistemas Java feitos com Spring Boot. Ele permite controlar quem pode acessar o sistema e o que cada usuário pode fazer, além de proteger a aplicação contra ataques comuns, como acesso indevido e requisições maliciosas.

2. Para que serve o método `csrf().disable()`?

R: O CSRF é uma proteção ao tipo de ataque onde alguém tenta enviar requisições em nome de um usuário autenticado sem que ele perceba. O método `csrf().disable()` serve para desativar a proteção contra CSRF no Spring Security, pois em APIs REST cada requisição já envia o token de autenticação.

3. Explique o que significa `SessionCreationPolicy.STATELESS`.

R: Significa que a aplicação não irá criar nem armazenar sessões do usuário no servidor. Cada requisição precisa conter todas as informações de autenticação, normalmente através de um token JWT.

4. Qual a função do `permitAll()`?

R: Está método libera o acesso a determinada rota sem exigir autenticação.

5. Explique o que faz o `requestMatchers()`.

R: Este método serve para definir quais rotas terão regras específicas de segurança, como por exemplo, qual usuário pode acessar está rota.

6. Qual a função do `anyRequest().authenticated()`?

R: Está método restringe que qualquer requisição que não tenha sido configurada anteriormente, com o `permitAll` e o `requestMatchers`, precisará de autenticação para ser acessada.

7. O que é uma API Stateless?

R: Uma API Stateless é uma API que não guarda informações de sessão do usuário no servidor. Cada requisição é independente e deve conter todos os dados necessários para autenticação e processamento.

8. Explique o que a lambda abaixo faz:

`session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS)`

R: Essa lambda configura o Spring Security para utilizar política de sessão stateless, que diz que o servidor não deve criar sessões HTTP e que cada requisição deve ser autenticada individualmente.